CH-6 Part-3

In-Depth Study Notes: Oracle SQL Non-Equijoins, Deep Chaining, and Data Auditing

As an expert Oracle SQL Developer, writing queries for deeply normalized schemas requires mastery over advanced relationship mapping and data auditing. Based on the execution logs provided (CH6-5.TXT and CH6-6.TXT), these notes cover **Non-Equijoins**, **Deep Multi-Table Chaining**, **Outer Joins**, and **Anti-Joins** (Data Auditing).

Concept 1: Non-Equijoins (Range Joins)

Core Purpose:A **Non-Equijoin** establishes a relationship between two tables without using an exact equality operator (=). Instead, it links rows by evaluating if a numeric or date value from the first table falls within a defined range (upper and lower bounds) stored in the second table 1, 2.

Keyword Breakdown:

- **JOIN ... ON**: The only join clause that supports Non-Equijoins. (NATURAL JOIN and JOIN ... USING strictly enforce equality matching).
- **BETWEEN ... AND**: A logical operator that evaluates if a value falls inclusively within a range 1. *Why use it over alternatives?* You can functionally achieve the exact same result using >= AND <= 3, but BETWEEN is the enterprise standard because it drastically reduces code verbosity and improves readability.

Syntax Breakdown:

```
SELECT column_list
FROM table1
JOIN table2 ON table1.numeric_col BETWEEN table2.lower_bound AND
table2.upper_bound;
```

Detailed Example: In the HR Schema, employees do not have a hardcoded GRADE_LEVEL. Their salary must be dynamically mapped against the JOB_GRADES tier list:

```
SELECT FIRST_NAME, SALARY, GRADE_LEVEL
FROM EMPLOYEES
JOIN JOB_GRADES ON SALARY BETWEEN LOWEST_SAL AND HIGHEST_SAL;
```

Execution outputs 20 rows, accurately categorizing salaries like 24000 to Grade E and 2500 to Grade A 1, 2.

Execution Logic:

1. **Data Loading:** The engine loads EMPLOYEES and JOB_GRADES into memory.
2. **Row-by-Row Iteration:** It takes the first employee's salary and sequentially evaluates the BETWEEN logic against every row in the JOB_GRADES table.
3. **Boolean Check:** If the logic evaluates to TRUE, the tables are joined for that specific row and projected.

Pro-Tips/Best Practices:

- **Data Architecture:** Ensure the upper and lower bounds in your lookup table (e.g., JOB_GRADES) **do not overlap**. If Tier 1 is 1000-2999, Tier 2 must start at 3000 4. If ranges overlap, a single employee will successfully join to multiple tiers, artificially inflating your query results with Cartesian duplicates.

Concept 2: Deep Multi-Table Chaining

Core Purpose: Deep chaining reconstructs a complete business object from a deeply normalized database. It involves seamlessly stacking JOIN statements to traverse the schema hierarchy (e.g., mapping an Employee to their Department, to the Location, to the Country, to the Region) 5, 6.

Keyword Breakdown:

- **Sequential JOIN clauses**: You can freely hybridize INNER JOIN ... ON, INNER JOIN ... USING, and NATURAL JOIN to bridge the tables, provided you obey the strict aliasing rules for each 5, 6.

Syntax Breakdown:

```
SELECT column_list
FROM table1
JOIN table2 USING (shared_pk)
JOIN table3 ON table2.val = table3.val
NATURAL JOIN table4;
```

Detailed Example: To retrieve a massive matrix mapping 19 employees across 7 distinct tables (Employees, Jobs, Job Grades, Departments, Locations, Countries, and Regions):

```
SELECT FIRST_NAME, DEPARTMENT_NAME, CITY, COUNTRY_NAME, REGION_NAME,
JOB_ID, JOBS.JOB_TITLE, SALARY, GRADE_LEVEL
FROM EMPLOYEES E
INNER JOIN JOBS USING (JOB_ID)
INNER JOIN JOB_GRADES ON SALARY >= LOWEST_SAL AND SALARY <=
HIGHEST_SAL
INNER JOIN DEPARTMENTS D ON D.DEPARTMENT_ID = E.DEPARTMENT_ID
INNER JOIN LOCATIONS L USING (LOCATION_ID)
INNER JOIN COUNTRIES USING (COUNTRY_ID)
INNER JOIN REGIONS USING (REGION_ID);
```

Execution perfectly maps the geographic and job profile for all assigned employees 5, 7-9.

Execution Logic: The Oracle Engine evaluates the joins sequentially (or as optimized by the Cost-Based Optimizer). It links EMPLOYEES to JOBS, taking that virtual result set and joining it to JOB_GRADES, and cascades this logic through the geographic tables until the full row is constructed.

Pro-Tips/Best Practices:

- **Typographical Strictness:** The Oracle compiler is unforgiving in long chains. Typing NATURAL JON instead of JOIN immediately triggers an **ORA-00933: SQL command not properly ended** error 10.
- **Alias Rules in USING:** When combining multiple tables, remember that any column placed in a USING clause cannot be qualified with an alias in the SELECT list.

Attempting to qualify it (e.g., COUNTRIES.COUNTRY_NAME in a natural/using context) triggers an **ORA-00904: invalid identifier** 11.

Concept 3: OUTER JOINS (LEFT, RIGHT, FULL)

Core Purpose: Standard INNER JOINS drop data if a match is not found. An **OUTER JOIN** solves this by adding unmatched/extra rows back into the final result set 12, 13. It ensures that primary data remains visible even if its related child data is missing or NULL.

Keyword Breakdown:

- **LEFT OUTER JOIN**: Keeps all matched rows PLUS unmatched rows from the table on the *left* side of the keyword 14.
- **RIGHT OUTER JOIN**: Keeps all matched rows PLUS unmatched rows from the table on the *right* side of the keyword 14, 15.
- **FULL OUTER JOIN**: Keeps matched rows PLUS all unmatched rows from *both* tables 16.
- **OUTER**: This keyword is optional. RIGHT JOIN and RIGHT OUTER JOIN parse identically 15, 17.

Syntax Breakdown:

```
SELECT column_list
FROM table1
LEFT|RIGHT|FULL OUTER JOIN table2 ON table1.col = table2.col;
```

Comparison of Outer Join Results:

Join Direction,Unmatched Left Rows,Unmatched Right Rows,Null Padding Behavior
LEFT JOIN,Kept,Dropped,Right-table columns padded with NULL
RIGHT JOIN,Dropped,Kept,Left-table columns padded with NULL
FULL JOIN,Kept,Kept,Padded on whichever side is missing the data

Detailed Example: To retrieve all Regions and their Countries, ensuring that empty Regions (like 'Asia' or 'Middle East and Africa' which have no mapped countries) are still displayed:

```
SELECT *
FROM COUNTRIES C
RIGHT JOIN REGIONS R ON C.REGION_ID = R.REGION_ID;
```

Execution outputs Canada, Germany, UK, USA, plus the missing Regions 3 and 4, padding their Country Name with NULL 17.

Execution Logic:

1. **Inner Match:** The engine evaluates C.REGION_ID = R.REGION_ID.
2. **Outer Sweep:** Because it is a RIGHT JOIN, it reviews the REGIONS table. It identifies that Regions 3 and 4 did not survive the inner match.
3. **Forced Projection:** It forces Regions 3 and 4 into the result set, filling the COUNTRY_NAME column with NULL 15, 17.

Concept 4: Data Auditing & Anti-Joins (IS NULL)

Core Purpose: While an Outer Join returns both matched and unmatched rows, an **Anti-Join** leverages a WHERE clause to filter out the successful matches, leaving **only the orphaned or**

extra rows. This is the industry-standard architecture for finding missing data or unassigned resources.

Keyword Breakdown:

- **WHERE ... IS NULL**: Explicitly isolates the dynamically generated NULL values created by the Outer Join. *Why use it?* You **must** use IS NULL. Using standard mathematical operators like = NULL fails because a NULL state is technically unknown and cannot be evaluated mathematically.

Syntax Breakdown:

```
SELECT column_list
FROM primary_table
LEFT JOIN secondary_table ON primary_table.pk = secondary_table.fk
WHERE secondary_table.fk IS NULL;
```

Detailed Example 1 (Equijoin Auditing): To display *only* the specific employee who has no assigned Department, and the specific Department that has no assigned employees:

```
SELECT FIRST_NAME , DEPARTMENT_NAME
FROM EMPLOYEES
FULL JOIN DEPARTMENTS USING (DEPARTMENT_ID)
WHERE DEPARTMENT_ID IS NULL OR EMPLOYEE_ID IS NULL;
```

Execution correctly filters out 19 mapped employees, leaving only 'Kimerely' (no department) and 'Contracting' (no employees) 18.

Detailed Example 2 (Non-Equijoin Auditing): To audit the JOB_GRADES tier list and find which salary bracket currently contains *zero* active employees:

```
SELECT GRADE_LEVEL
FROM EMPLOYEES
RIGHT JOIN JOB_GRADES ON SALARY BETWEEN LOWEST_SAL AND HIGHEST_SAL
WHERE EMPLOYEE_ID IS NULL;
```

Execution returns 'F', as no employee currently earns between 25000 and 40000 4, 19.

Execution Logic:

1. The complete Outer Join executes in memory, mapping rows and padding orphans with NULL.
2. The WHERE clause runs sequentially *after* the join phase.
3. Any row with a valid EMPLOYEE_ID (meaning the match was successful) is discarded. Only the orphaned rows pass the IS NULL boolean check and are projected.

Pro-Tips/Best Practices:

- **Performance Warning:** Filtering by IS NULL usually forces a **Full Table Scan**, as standard Oracle B-Tree indexes do not index NULL values. If this Anti-Join runs frequently on massive tables in production, consider implementing Bitmap indexes or Function-Based indexes to optimize the scan.